PYTHON PROGRAMMING MINI PROJECT

STUDENT RECORD MANAGEMENT SYSTEM

Project Report

Submitted by
Student Name: Sanjana Dattatraya Gaikwad Roll No.: 21
Department: School of Engineering and Technology
Class / Division: SY BTech, Div C
Subject: Python Programming
Submitted to: Ashish Mishra
Academic Year: 2026–2027

A menu-driven Python application for storing and managing student records.

1. Introduction

Educational institutions need to maintain student information such as roll number, name, course or class, and marks. Keeping these details manually can take time and may lead to duplicate or misplaced records. The Student Record Management System is a simple, menu-driven Python program that helps a user maintain student information from the console.

The program can add a new student, display all records, search by roll number, update details, delete a record, and save information in a local text file for use the next time the program starts.

2. Problem Statement

Maintaining student records manually can make it difficult to find a particular student, correct existing information, or keep a consistent list. This project provides a simple way to add, view, search, modify, delete, and save student records using basic Python programming.

3. Objectives

• Store each student's roll number, name, course/class, and marks.

• Display all student records in a readable format.

• Search for a student using a roll number.

• Update existing student details and delete unwanted records.

• Save records in a text file and load them at startup.

• Validate user input and prevent duplicate roll numbers.

• Apply Python programming concepts in a practical mini project.

4. Scope of the Project

This is a beginner-level, console-based application intended for practice and small demonstrations. Records are stored locally in student_records.txt. It does not require a database, web server, graphical interface, or external Python library. It is not intended to replace a secure institutional student information system.

5. Features

Feature	Description
Add Student	Accepts roll number, name, course/class, and marks.
View All Students	Displays all stored student records in columns.
Search Student	Finds a record using its roll number.
Update Student	Allows the user to change name, course/class, or marks.
Delete Student	Removes a selected record by roll number.
Save and Load	Writes records to student_records.txt and loads them on the next run.
Input Validation	Checks empty/duplicate roll numbers and marks from 0 to 100.

6. Python Concepts Used

Concept	Use in the project
Variables and data types	Store roll numbers, names, courses, marks, and choices.
Lists and dictionaries	Keep multiple student records in memory.
Input/output	input() collects details and print() displays results.
Loops	while repeats the menu; for processes records.
Conditional statements	if-elif-else selects operations and validates data.
Functions	Separate tasks into add_student(), search_student(), and save_records().
File handling	Read and write records in student_records.txt.
Exception handling	Handles invalid marks and file errors.

7. Methodology

• At startup, load_records() reads saved records from student_records.txt if it exists.

• The menu displays six options: Add, View, Search, Update, Delete, and Exit.

• The selected option calls its corresponding function.

• When adding a record, the program checks for a non-empty, unique roll number and valid marks from 0 to 100.

• Records are stored in a list of dictionaries and written to the text file.

• The menu repeats until the user chooses Exit.

8. Algorithm

• Start and load saved student records.

• Display the menu and read the user's choice.

• For option 1, add a student and save the record.

• For option 2, display all records.

• For option 3, search by roll number.

• For option 4, update an existing record.

• For option 5, delete a record.

• For option 6, display the exit message and stop.

• For an invalid choice, show an error and repeat the menu.

9. Source Code Screenshots

The following screenshot-style images show the Python source code in consecutive sections. The editable source file is provided separately as student_record_management_system.py.

Source Code Screenshot 1

student_record_management_system.py — Code Screenshot 1/6

```
001  import os
002
003  FILE_NAME = "student_records.txt"
004  students = []
005
006  def load_records():
007      """Load saved student records from the text file."""
008      if not os.path.exists(FILE_NAME):
009          return
010      try:
011          with open(FILE_NAME, "r", encoding="utf-8") as file:
012              for line in file:
013                  parts = line.rstrip("\n").split("|")
014                  if len(parts) == 4:
015                      roll, name, course, marks = parts
016                      students.append({"roll": roll, "name": name,
017                                       "course": course, "marks": marks})
018      except OSError:
019          print("Unable to read the records file.")
020
021  def save_records():
022      """Save all student records to the text file."""
023      try:
024          with open(FILE_NAME, "w", encoding="utf-8") as file:
025              for s in students:
026                  file.write(f'{s["roll"]}|{s["name"]}|{s["course"]}|{s["marks"]}\n')
027      except OSError:
```

Source Code Screenshot 2

student_record_management_system.py — Code Screenshot 2/6

```
        print("Unable to save the records.")

def find_student(roll):
    for student in students:
        if student["roll"] == roll:
            return student
    return None

def add_student():
    roll = input("Enter roll number: ").strip()
    if not roll:
        print("Roll number cannot be empty.")
        return
    if find_student(roll):
        print("A student with this roll number already exists.")
        return
    name = input("Enter student name: ").strip()
    course = input("Enter course/class: ").strip()
    if not name or not course:
        print("Name and course/class cannot be empty.")
        return
    try:
        marks = float(input("Enter marks (0-100): "))
        if not 0 <= marks <= 100:
            print("Marks must be between 0 and 100.")
            return
    except ValueError:
```

Source Code Screenshot 3

student_record_management_system.py — Code Screenshot 3/6

```
        print("Please enter marks as a number.")
        return
    students.append({"roll": roll, "name": name, "course": course, "marks": str(marks)})
    save_records()
    print("Student record added successfully.")

def view_students():
    if not students:
        print("No student records found.")
        return
    print("\n{:<12} {:<24} {:<18} {:>8}".format(
        "Roll No.", "Student Name", "Course/Class", "Marks"))
    print("-" * 66)
    for s in students:
        print("{:<12} {:<24} {:<18} {:>8}".format(
            s["roll"], s["name"], s["course"], s["marks"]))

def search_student():
    roll = input("Enter roll number to search: ").strip()
    s = find_student(roll)
    if s:
        print("\nStudent record found:")
        print("Roll No.    :", s["roll"])
        print("Name        :", s["name"])
        print("Course/Class:", s["course"])
        print("Marks       :", s["marks"])
    else:
```

Source Code Screenshot 4

student_record_management_system.py — Code Screenshot 4/6

```
        print("No student found with that roll number.")

def update_student():
    roll = input("Enter roll number to update: ").strip()
    s = find_student(roll)
    if not s:
        print("No student found with that roll number.")
        return
    name = input(f'New name [{s["name"]}] (Enter to keep): ').strip()
    course = input(f'New course [{s["course"]}] (Enter to keep): ').strip()
    marks_text = input(f'New marks [{s["marks"]}] (Enter to keep): ').strip()
    if name:
        s["name"] = name
    if course:
        s["course"] = course
    if marks_text:
        try:
            marks = float(marks_text)
            if not 0 <= marks <= 100:
                print("Marks must be between 0 and 100.")
                return
            s["marks"] = str(marks)
        except ValueError:
            print("Please enter marks as a number.")
            return
    save_records()
    print("Student record updated successfully.")
```

Source Code Screenshot 5

student_record_management_system.py — Code Screenshot 5/6

```
001
002 def delete_student():
003     roll = input("Enter roll number to delete: ").strip()
004     s = find_student(roll)
005     if not s:
006         print("No student found with that roll number.")
007         return
008     students.remove(s)
009     save_records()
010     print("Student record deleted successfully.")
011
012 def main():
013     load_records()
014     while True:
015         print("\n===== STUDENT RECORD MANAGEMENT SYSTEM =====")
016         print("1. Add Student")
017         print("2. View All Students")
018         print("3. Search Student")
019         print("4. Update Student")
020         print("5. Delete Student")
021         print("6. Exit")
022         choice = input("Enter your choice (1-6): ").strip()
023         if choice == "1":
024             add_student()
025         elif choice == "2":
026             view_students()
027         elif choice == "3":
```

Source Code Screenshot 6

student_record_management_system.py — Code Screenshot 6/6

```
            search_student()
        elif choice == "4":
            update_student()
        elif choice == "5":
            delete_student()
        elif choice == "6":
            print("Thank you for using Student Record Management System.")
            break
        else:
            print("Invalid choice. Please select 1-6.")

if __name__ == "__main__":
    main()
```

10. Sample Output Screenshots

These sample console outputs illustrate the main operations. Example student names and marks are used for demonstration.

Sample Output Screenshot 1: Add Student

Terminal — Sample Output 1: Add Student

```
===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice (1-6): 1
Enter roll number: 21
Enter student name: Sanjana Gaikwad
Enter course/class: SY BTech
Enter marks (0-100): 86
Student record added successfully.
```

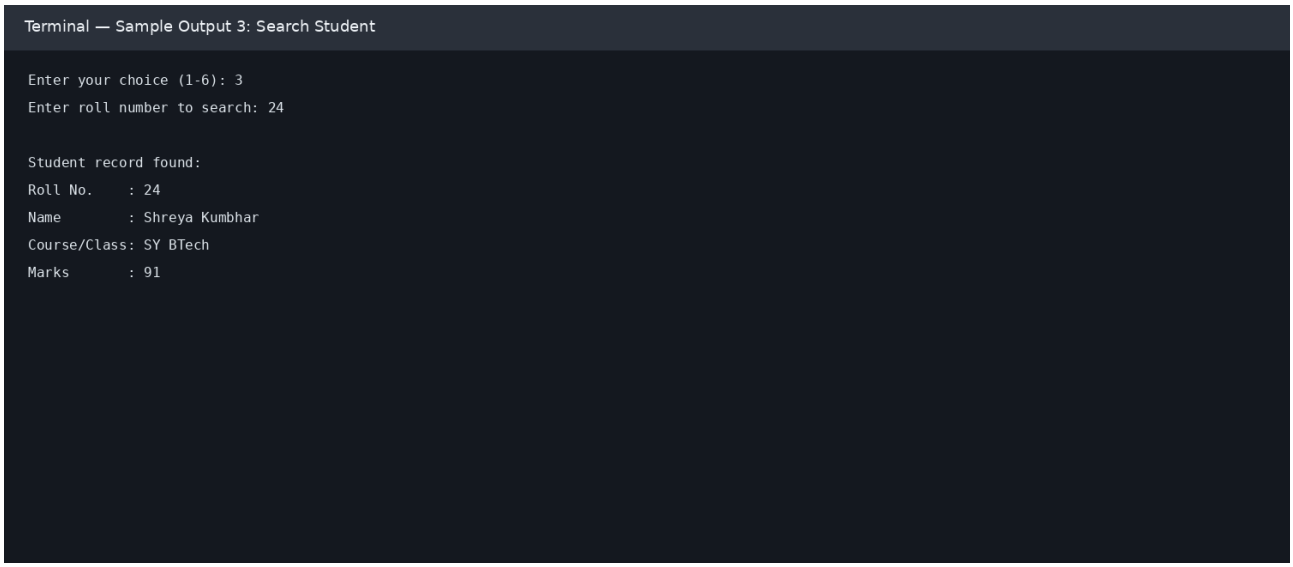
Sample Output Screenshot 2: View All Students

Terminal — Sample Output 2: View All Students

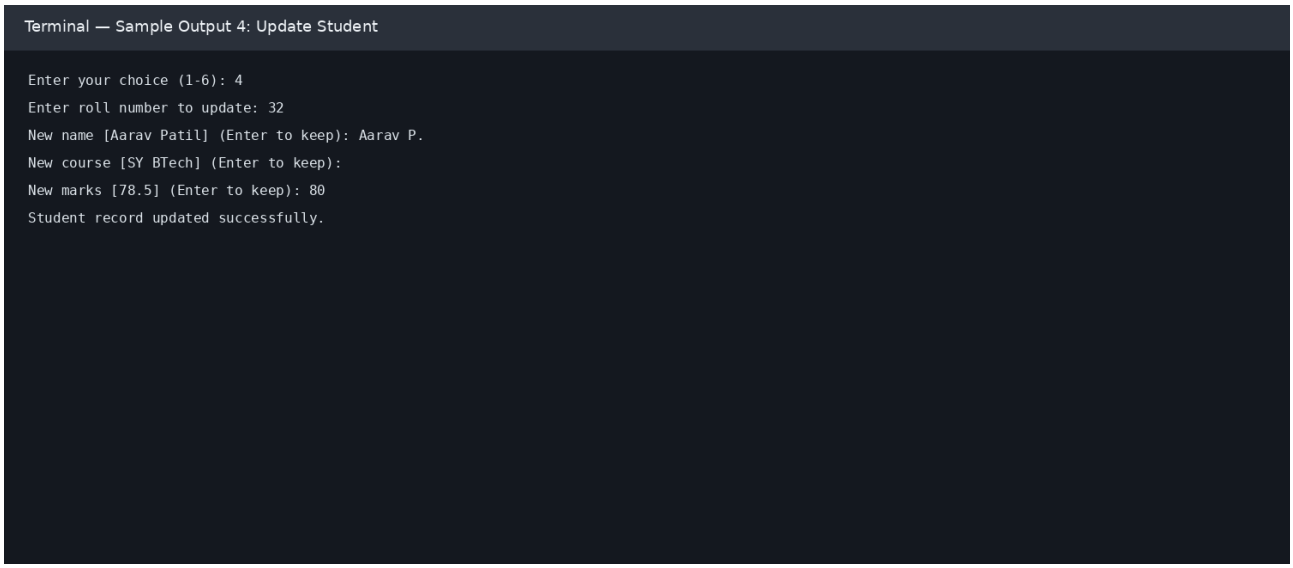
```
Enter your choice (1-6): 2

Roll No.     Student Name            Course/Class         Marks
------------------------------------------------------------------
21           Sanjana Gaikwad          SY BTech               86
24           Shreya Kumbhar           SY BTech               91
32           Aarav Patil              SY BTech             78.5
```

Sample Output Screenshot 3: Search Student



Sample Output Screenshot 4: Update Student



Sample Output Screenshot 5: Delete Student

Terminal — Sample Output 5: Delete Student

```
Enter your choice (1-6): 5
Enter roll number to delete: 32
Student record deleted successfully.
```

Sample Output Screenshot 6: Validation and Exit

Terminal — Sample Output 6: Validation and Exit

```
Enter your choice (1-6): 1
Enter roll number: 21
A student with this roll number already exists.

Enter your choice (1-6): 9
Invalid choice. Please select 1-6.

Enter your choice (1-6): 6
Thank you for using Student Record Management System.
```

11. Testing

Test	Expected result
Add valid student	Record is saved and success message appears.
View records	All stored students are displayed.
Search existing roll number	Matching student's details are displayed.
Search unknown roll number	No-student-found message appears.
Update record	Selected fields are updated and saved.
Delete record	Selected student is removed and file is updated.
Enter duplicate roll number	Program rejects the duplicate.
Enter invalid marks	Validation message is displayed.
Exit	Thank-you message appears and program stops.

12. GitHub Repository

Upload student_record_management_system.py to a GitHub repository and add the repository URL below.

GitHub Repository Link: _____

13. Conclusion

The Student Record Management System is a simple Python mini project for maintaining basic student details. It supports adding, viewing, searching, updating, and deleting records, and stores data in a text file for future use. It demonstrates functions, lists, dictionaries, loops, conditions, validation, exception handling, and file operations. Future versions could add a graphical interface, database connectivity, or more student fields.

How to run: Run `python student_record_management_system.py` in a terminal. The program creates student_records.txt when records are saved.